SIMATS ENGINEERING

Department of Information Technology

ITA05 – COMPUTER VISION

ASSIGNMENT 4 – CO4

Compare, Justify and Engineer Object and Pattern Recognition Techniques for Pedestrian-Crossing Safety at a College Gate

CO4: Compare object and pattern recognition techniques for interpreting visual data with spatial and motion characteristics.

Total Marks: 50

Duration: 1 week

STUDENT DETAILS

Particulars	Details	Particulars	Details
Student Name	M. Sahasra	Register Number	192425317
Department / Section	Artificial Intelligece & Machine Learning	Academic Year / Semester	2025-2026
Faculty Name	Dr. Bindu G	Nature of Assignment	Individual
Duration	1 Week	Total Marks	50
Submission Date	_____	Course Code	ITA05

Assignment Title: Compare, Justify and Engineer Object and Pattern Recognition Techniques for Pedestrian-Crossing Safety at a College Gate

MARK SHEET (Rubric-based Evaluation)

Task	Topic	Marks	Marks Awarded	Remarks
1	Viola-Jones Detection	4		
2	YOLO and Deep Learning Detectors	7		
3	Background Modeling Techniques	5		
4	Optical Flow and Motion Estimation	6		
5	Stereo Vision and Structure from Motion	7		
6	Pattern Recognition Classifiers	4		
7	Motion Models for Image Animation	5		
8	Integrated Pipeline Evaluation	4		
9	System Re-evaluation and Redesign	8		
TOTAL		50		

OVERALL REMARKS

Student Signature	Faculty Signature	Program Director

MANDATORY SUBMISSION EVIDENCE CHECK

✓ = present ✗ = missing / not satisfied P = partly present. Based on the "Evidence Expected" of each task. Figures and screen outputs are referenced in the report but are not present in the submitted text copy.

Task	Evidence expected	Status	Notes
1	Annotated detector outputs, Haar/LBP/strong-classifier comparison, detection-rate and false-positive results, and justified selection.		
2	YOLO/SSD/Faster R-CNN outputs, labelled detection results, precision, recall, AP/mAP@0.5, FPS and small-object/occlusion analysis.		
3	GMM component-match calculations and update steps, background-model comparison, foreground masks and parameter analysis.		
4	Optical-flow derivation, Lucas–Kanade/Horn–Schunck/block-matching comparison, motion vectors and speed calculation.		
5	Stereo depth calculations for all disparities, disparity/depth maps, block matching/SGM/SfM comparison and baseline sensitivity calculation.		
6	Extracted image/blob features, k-NN/SVM/decision-tree comparison, confusion matrix and class-wise precision/recall.		
7	Animated outputs, translation/affine/keypoint motion comparison, motion-field/occlusion evidence and visual-quality assessment.		

8	Detection-only/motion-first/hybrid pipeline comparison, ablation results, FPS, speed/depth error and robustness analysis.		
9	Redesigned junction architecture, 200 ms timing budget, risk/false-alarm analysis, night/rain adaptation and final design decision.		

IMPLEMENTATION EVIDENCE CHAIN (Tasks 1–10)

Stage	Evidence to retain	Used in tasks
Image and video acquisition	Original images/videos, camera settings, frame rate, resolution and lighting conditions.	1–9
Image preprocessing	Resized/grayscale frames, denoising, normalization and preprocessing parameters.	1, 2, 3, 4, 5, 6, 8, 9
Object detection	Haar/LBP/strong-classifier and YOLO-family bounding boxes, confidence scores and detection logs.	1, 2, 8, 9
Background modelling	GMM parameters, foreground masks, component matches and update records.	3, 8, 9
Motion estimation	Optical-flow vectors, tracked points, displacement and speed calculations.	4, 7, 8, 9
Depth estimation	Stereo image pairs, disparity maps, depth values and calibration parameters.	5, 8, 9
Feature extraction	Blob/contour measurements, area, aspect ratio, height, solidity and speed features.	6, 8, 9
Classification	Predicted object classes, confidence, confusion matrix and class-wise metrics.	6, 8, 9
Motion transfer	Keypoints, affine transforms, dense motion fields, occlusion maps and generated frames.	7

FINAL SUBMISSION EVIDENCE CHECK

- ☐ Prediction-versus-observation table completed for each task.
- ☐ Task 1–2 detection comparison includes per-class metrics and failure images.
- ☐ Task 3 hand calculations and background-model experiment are included.
- ☐ Task 4 optical-flow derivation, speed calculation and tracking comparison are included.
- ☐ Task 5 stereo depth calculations, known-distance validation and SfM scale discussion are included.
- ☐ Task 6 confusion matrices and cost-sensitive pedestrian decision are included.
- ☐ Task 7 three animation outputs are labelled synthetic and consent safeguards are documented.
- ☐ Task 8 pipeline comparison and ablation study are completed.
- ☐ Task 9 time budget, night/rain test, false-alarm rate, revised pseudocode and final decision are included.
- ☐ All results are reproducible and clearly labelled as measured, calculated, estimated or illustrative.

IMPLEMENTATION EVIDENCE CHAIN (Tasks 1–9)

Stage	Input	Output / Evidence
1. Detection	Annotated CCTV frames	Pedestrian / two-wheeler / auto / car / bus bounding boxes
2. Background modeling	Video frames	Foreground masks and precision/recall
3. Motion estimation	Consecutive frames	Optical-flow vectors, displacement and speed
4. Depth recovery	Stereo pair / moving clip	Disparity, depth map, known-distance errors
5. Pattern classification	Blob and motion features	Class labels, confusion matrix
6. Animation	Still image + measured motion	Synthetic frames, warp/occlusion artifact analysis

7. Integration	Detection, motion and depth	Pipeline comparison, ablation and timing
8. Redesign	Night/rain/single camera	Latency budget, alarm rule and final design

COMPUTER VISION

ASSIGNMENT 4 – CO4

CO4: Compare object and pattern recognition techniques for interpreting visual data with spatial and motion characteristics.

Nature of Assignment: Individual

Duration: 1 Week

Total Marks: 50

Submission Date: _____

SCENARIO AND OPERATING REQUIREMENTS

The college gate is beside a busy road. The safety system must detect pedestrians and vehicles, distinguish moving objects from the background, estimate vehicle speed, recover depth, classify objects from appearance and motion, and support a clearly labelled synthetic reconstruction of a near-miss event.

Important difficulties include small distant pedestrians, partial occlusion, shadows, swaying trees, dusk/night illumination and vehicle categories such as auto-rickshaws that may be absent from pretrained label sets. Missing a pedestrian is a serious safety risk, while excessive false alarms can cause alarm fatigue.

DATA AND TOOLS

- Fixed-camera video: at least 60 seconds at 25 fps or higher, containing pedestrians, two-wheelers, auto-rickshaws and cars, plus at least 15 seconds at dusk or night.
- Stereo image pair or short sequence with known focal length in pixels and baseline; road markings have a known spacing such as 5 m.
- Moving-camera clip of at least 10 seconds for structure from motion and one still photograph for Task 7.
- Ground truth: bounding boxes and class labels in at least 50 frames; manually tracked positions of at least two objects over 10 frames; known distances for at least three stereo-scene points.
- Any language/tool may be used. Library calls are permitted only when parameters, thresholds, decision rules and evaluation are defined and justified.

- Algorithm design tasks require language-independent pseudocode following the SIMATS Handbook for Algorithmic Pseudocode.

EXPERIMENTAL SETUP AND METRIC DEFINITIONS

Metric	Meaning / Formula
Precision	TP / (TP + FP): fraction of predicted positives that are correct.
Recall / Detection rate	TP / (TP + FN): fraction of ground-truth objects detected.
IoU	Area of intersection / area of union of predicted and ground-truth boxes.
AP / mAP	Area under precision–recall curve; mAP is the mean AP over evaluated classes.
FPS	Processed frames / elapsed seconds; latency per frame is 1 / FPS.
False positives per frame	Number of unmatched detections divided by number of evaluated frames.
Optical-flow displacement	Pixel displacement between two frames; convert to physical speed only with a valid image-to-world scale.
Stereo depth	$Z = fB / d$, where f is focal length, B is baseline and d is disparity.
Depth error	Estimated depth minus known depth; report absolute error and preferably MAE/RMSE.
Confusion matrix	Counts of true classes versus predicted classes; derive per-class precision and recall.
False alarms per hour	Number of alarm events divided by monitored hours; merge consecutive-frame detections into one event.

TASK 1 – Viola-Jones Detection: Compare Three Design Variants (4 Marks)

Question / Required Work

Compare Haar cascade (full Viola–Jones), LBP cascade and a single strong classifier without a cascade. Explain Haar-like features, integral image, AdaBoost and cascade operation; compare detection rate, false positives per frame and time per frame for pedestrians, two-wheelers and cars; compare with a lightweight modern detector.

Compare object recognition techniques (Viola-Jones) spatial characteristics (appearance features)

Technique	**Basic idea and key OpenCV functions**
A. Haar cascade (full Viola-Jones)	Haar-like features, integral image, AdaBoost feature selection and an attentional cascade with early rejection.
B. LBP cascade	The same cascade structure but with Local Binary Pattern features, which are cheaper to compute.
C. Single strong classifier (no cascade)	One boosted classifier combining all selected features, evaluated fully on every window in a sliding-window scan.

- Analyze how Haar-like features, the integral image, AdaBoost and the cascade work together. Show why any rectangle sum needs only four look-ups in the integral image, whatever the rectangle size.
- Predict, then compare A, B and C on pedestrians, two-wheelers and cars in your annotated frames: detection rate, false positives per frame and time per frame for each class.
- Explain from the nature of the features which class each variant handles best and worst.
- Justify with evidence which variant you would choose for the gate.
- Compare Viola-Jones with a current lightweight detector (for example, YOLO11-n or MobileNet-SSD) on the same clip: accuracy and time per frame on a CPU.
- Evidence Expected: A three-variant comparison table per class (detection rate, false positives, time), predicted-versus-observed comparison, and an explanation in terms of the features used.

Answer

1.1 Working of the Viola–Jones framework

Viola–Jones is a sliding-window object detector. It evaluates simple intensity-contrast patterns called Haar-like features. A feature compares sums of pixels in adjacent rectangular regions, for example a two-rectangle edge feature or a three-rectangle line feature. These features describe local appearance rather than semantic meaning.

The integral image $I\Sigma(x,y)$ stores the sum of all pixels above and to the left of (x,y). The sum of any axis-aligned rectangle can be calculated from four corner look-ups: $S = I\Sigma(x2,y2) - I\Sigma(x1-1,y2) - I\Sigma(x2,y1-1) + I\Sigma(x1-1,y1-1)$. Therefore, the number of look-ups is constant and does not depend on rectangle area. This makes repeated feature evaluation fast.

AdaBoost selects a small set of useful features from a very large candidate pool and combines weak classifiers into a stronger classifier. The cascade arranges classifiers in stages. Easy background windows are rejected in early stages; only promising windows reach later, more expensive stages. This improves speed while aiming to retain high detection recall.

1.2 Comparison of the three variants

Variant A uses Haar-like rectangular contrast features with the integral image, AdaBoost and a cascade. It can detect fairly rigid, front-facing patterns but is sensitive to pose, scale, lighting and occlusion. Variant B uses Local Binary Pattern features, which encode local texture and are computationally efficient; it can be useful when texture is stable but may struggle with low-texture or strongly changing appearance. Variant C evaluates a single strong classifier at every window. It avoids early-stage rejection but usually performs more computations on background windows.

1.3 Expected class behaviour

Pedestrians are relatively deformable and may be partly hidden; a trained pedestrian cascade may detect upright people but can miss unusual poses and distant small people. Two-wheelers have varied shapes and rider/vehicle overlap, so a simple cascade may confuse them with background or other vehicles. Cars have more consistent rectangular appearance and are often easier for classical cascades, although perspective and occlusion still cause errors. LBP may be efficient on textured regions; Haar can capture coarse shape/contrast; a no-cascade classifier is generally slower.

1.4 Selection for the gate

A cascade is preferable to the single strong classifier when CPU speed is important because early rejection avoids evaluating every feature on most windows. However, a missed pedestrian is costly. A classical cascade should therefore be treated as a baseline or proposal generator, not as the sole safety detector. A lightweight modern detector such as YOLO11-n or MobileNet-SSD is a stronger practical candidate because it learns more varied appearance patterns. Select the final detector only after testing the same frames and reporting pedestrian recall, false alarms and CPU latency.

1.5 Integral-image rectangle example

For a rectangle with corners A=(x1−1,y1−1), B=(x2,y1−1), C=(x1−1,y2), D=(x2,y2), the pixel sum is D − B − C + A. The four terms are retrieved from the integral image. This is O(1) per rectangle feature, independent of the rectangle's width and height.

Three-Technique Comparison

Technique	Strengths	Limitations / likely failure	Best role
Haar cascade	Fast rectangle features; cascade rejects easy background windows early.	Sensitive to pose, scale, occlusion and illumination; requires suitable training samples.	Baseline detector for relatively rigid, frontal objects.
LBP cascade	Compact local texture coding; often computationally inexpensive.	Texture may change with blur, distance, shadows and low resolution.	Fast texture-based baseline; compare on each object class.
Single strong classifier	Uses all selected weak learners on every candidate window.	No early cascade rejection; higher scan cost and more background evaluations.	Useful as a controlled comparison, rarely the best real-time choice.

Evidence to Record and Decision

Create one result table per class with detection rate, false positives/frame and milliseconds/frame. Also record CPU comparison with a lightweight detector. Report predicted versus observed results. Do not claim a winner without testing the same annotated frames.

Experimental reporting note: Fill the result columns from the assigned video/images and actual program execution. Do not present predicted values as measured results.

TASK 2 – YOLO and Deep Learning Detectors: Compare, Justify and Adapt (7 Marks)

Question / Required Work

Compare YOLO, SSD and Faster R-CNN against the best Task 1 baseline. Explain YOLO's grid prediction, anchors, IoU and NMS; evaluate AP, mAP@0.5, precision, recall and FPS; analyze small objects, occlusion, dusk and domain shift; design a detector adaptation plan and compare CNN YOLO with RT-DETR or an open-vocabulary detector.

Marks Compare object recognition techniques (YOLO, deep learning algorithms) · spatial characteristics (appearance, size, position)

Technique	Basic idea and key OpenCV functions
A. YOLO	Single-pass grid detector: each cell predicts boxes, objectness and class scores; anchors, IoU and non-maximum suppression (NMS).
B. SSD	Single-shot detector using default boxes at several feature-map scales.
C. Faster R-CNN	Two-stage detector: a region proposal network followed by per-region classification and box refinement.

- Analyze the single-pass grid formulation of YOLO: what each cell predicts, the role of anchor boxes, IoU and NMS, and how SSD and Faster R-CNN differ in structure.
- Compare A, B and C (with your best Task 1 variant as a baseline row) on the same annotated frames: per-class average precision, mAP at IoU 0.5, precision, recall and frames per second.
- Analyze failures: small distant pedestrians, occlusion, dusk, and vehicle types missing from the pretrained label set (domain shift).
- Create: design a plan (pseudocode) to adapt your chosen detector to this junction: label scheme, whether to fine-tune, and the confidence threshold for each class chosen from the precision–recall curve, with the stated cost that a missed pedestrian is worse than a false alarm.
- Compare a CNN-based YOLO with a transformer-based detector (RT-DETR) or an open-vocabulary detector (Grounding DINO or YOLO-World) that can find “person with umbrella” without retraining.

Evidence Expected: *Comparison table for A, B, C (AP, mAP, precision, recall, fps), failure-case images with explanations, precision–recall curves with justified thresholds, and the adaptation pseudocode.*

Answer

2.1 YOLO detection process

YOLO performs object detection in one forward pass. Its feature maps predict candidate bounding boxes, objectness confidence and class scores at multiple spatial positions/scales. In anchor-based versions, predefined anchor shapes help predict boxes with different aspect ratios and sizes; anchor-free versions predict box geometry without fixed anchors. Objectness estimates whether a box contains an object, while class scores estimate its category.

IoU measures overlap between a predicted box and a ground-truth or another predicted box. During training it contributes to localization loss and matching. During inference, non-maximum suppression (NMS) sorts boxes by confidence and removes highly overlapping duplicates, usually within each class. The IoU threshold for NMS must be selected carefully: too low may suppress nearby objects; too high may retain duplicate boxes.

2.2 YOLO, SSD and Faster R-CNN

YOLO is a single-stage detector that predicts boxes and classes directly from feature maps. SSD is also single-stage and uses default boxes at several feature-map scales, helping detect objects of different sizes. Faster R-CNN is a two-stage detector: a Region Proposal Network generates candidate regions, then a second stage classifies and refines them. Faster R-CNN can provide strong localization but often has higher latency than compact single-stage models.

2.3 Evaluation and failure cases

Evaluate all systems on identical annotated frames. Report AP separately for pedestrian, two-wheeler, auto-rickshaw, car and bus; report mAP@0.5, precision, recall and FPS. Distant pedestrians occupy few pixels and may disappear after image resizing. Occlusion removes visible evidence and can split one object into fragments. Dusk lowers contrast and increases sensor noise. Domain shift occurs when a pretrained model has not learned the local vehicle type or class definition; an auto-rickshaw may be labelled as a car, motorcycle or unknown object.

2.4 Adaptation plan

Create a consistent label scheme with separate classes for pedestrian, two-wheeler, auto-rickshaw, car and bus. Annotate examples across distance, pose, day/dusk, partial occlusion and weather. Fine-tune a compact pretrained detector if the local classes are absent or poorly represented. Split training, validation and test data by video/time rather than randomly splitting adjacent frames, to reduce leakage. Choose class-specific confidence thresholds from validation precision–recall curves. Use a lower threshold for pedestrians when recall is the priority, then control false alarms using tracking and temporal confirmation.

2.5 CNN and transformer/open-vocabulary comparison

YOLO uses convolutional feature extraction and is often efficient for real-time deployment. RT-DETR uses a transformer-based detection design and can model global relationships, but actual CPU/embedded latency depends on model size and implementation. Grounding DINO or YOLO-World can use text prompts for open-vocabulary concepts such as 'person with umbrella', which can help identify categories not included in a fixed label set. Open-vocabulary flexibility does not guarantee reliable local classification; validate the prompts and output thresholds on the gate dataset.

Three-Technique Comparison

Technique	Strengths	Limitations / likely failure	Best role
YOLO	Single-stage; generally good speed/accuracy balance; multi-scale features.	Small distant people, domain shift and severe occlusion can reduce recall.	Strong primary detector after local fine-tuning.
SSD	Single-stage with default boxes at multiple scales; straightforward deployment.	Small-object recall may be weaker depending on backbone and input resolution.	Compact baseline for CPU comparison.

Faster R-CNN	Two-stage proposals and refinement can localize objects accurately.	Higher computational cost and potentially lower FPS.	Accuracy-focused benchmark, not automatically suitable for real time.

Algorithmic Pseudocode

```
ALGORITHM AdaptGateDetector
INPUT: labelled training frames, validation frames, test video
OUTPUT: trained detector and class-specific confidence thresholds

1. Define classes: pedestrian, two_wheeler, auto_rickshaw, car, bus.
2. Annotate boxes across distance, lighting, pose and occlusion conditions.
3. Split data by recording session into training, validation and test sets.
4. Load a compact pretrained detector.
5. If local classes are missing or poorly detected, fine-tune using training data.
6. Evaluate each class on validation data and generate precision-recall curves.
7. Select a pedestrian threshold that prioritizes high recall.
8. Select other class thresholds to balance precision and recall.
9. Run the detector on the held-out test video.
10. Report AP, mAP, precision, recall, FPS and class-wise errors.
11. Return detections with confidence and class labels.
END ALGORITHM
```

Evidence to Record and Decision

Report per-class AP, mAP@0.5, precision, recall and FPS. Include failure images and PR curves. Explain the cost-sensitive threshold choice. Compare with the Task 1 baseline on the same held-out frames.

Experimental reporting note: Fill the result columns from the assigned video/images and actual program execution. Do not present predicted values as measured results.

TASK 3 – Gaussian Mixture Model: Compare Background Modeling Techniques (5 Marks)

Question / Required Work

Calculate the GMM component match and updates for x=118 and x=200 using the given three-Gaussian model, $\alpha=\rho=0.05$, match test $|x-\mu|<2.5\sigma$ and background proportion $T=0.7$. Then compare GMM, running average and frame differencing and design a stopped-vehicle rule.

Compare object recognition techniques (GMM) · motion characteristics (separating moving objects from background)

Part (a) – Analyze by hand. The background model of one pixel is a mixture of three Gaussians:

Component	**Weight w**	**Mean μ**	**Std. deviation σ**
1	0.60	120	4
2	0.30	60	6
3	0.10	200	10

- A new pixel value x = 118 arrives. Use a match test of $|x - \mu| < 2.5\sigma$, background proportion T = 0.7 and learning rate $\alpha = 0.05$. Which component matches? Order the components by w/σ and decide how many are background. Is x background or foreground?
- Update the weights, and the mean and variance of the matched component (take $\rho = \alpha$). Repeat for x = 200 and explain the decision. What does it tell you that component 2 (mean 60) is classed as background?

Part (b) – Compare and create.

Technique	**Basic idea and key OpenCV functions**
A. GMM background model	Mixture of K Gaussians per pixel (e.g., K = 3) with online update; matching threshold, learning rate α and proportion T.
B. Running-average background	A single adaptive average per pixel with a fixed learning rate; foreground where the difference exceeds a threshold.

C. Frame differencing	Foreground where the difference between consecutive frames exceeds a threshold, with no background model.

- Compare A, B and C on your CCTV clip: foreground-mask precision and recall against your annotations, and behavior with shadows, swaying trees, camera shake, a stopped vehicle, dusk change and processing cost. Justify K, α, T and the matching threshold for technique A.
- Create: design a rule (pseudocode) that stops a vehicle waiting at a red light from being absorbed into the background, without making the model unable to adapt to dusk.
- Compare the GMM with a deep segmentation model for people and vehicles (for example, YOLO-seg) on shadows, swaying trees and a stopped vehicle.
- Evidence Expected: Hand-calculation table, three-technique comparison of mask precision/recall and failure behavior, justified parameters, failure images and the stopped-vehicle pseudocode.

Answer

3.1 Hand calculation: component matching for x = 118

The initial components are: C1 (w=0.60, μ=120, σ=4), C2 (w=0.30, μ=60, σ=6), C3 (w=0.10, μ=200, σ=10). The match condition is $|x-\mu| < 2.5\sigma$. For x=118: C1 difference=2 and limit=10, so it matches. C2 difference=58 and limit=15, so it does not match. C3 difference=82 and limit=25, so it does not match.

Rank components by w/σ: C1=0.60/4=0.150; C2=0.30/6=0.050; C3=0.10/10=0.010. The cumulative weights in this order are 0.60, 0.90 and 1.00. With T=0.7, C1 alone is insufficient, so C1 and C2 are treated as background components. Since x=118 matches C1, it is classified as background.

3.2 Update after x = 118

First decay all weights by $(1-\alpha)=0.95$, then add α=0.05 to the matched component: $w1=0.60\times0.95+0.05=0.62$; $w2=0.30\times0.95=0.285$; $w3=0.10\times0.95=0.095$. The weights sum to 1. For the matched component, $\mu1,new=\mu1+\rho(x-\mu1)=120+0.05(-2)=119.9$. Variance update: $\sigma^2new=(1-\rho)\sigma^2old+\rho(x-\mu old)(x-\mu new)=0.95\times16+0.05\times(-2)(-1.9)=15.39$. Thus $\sigma1,new\approx3.923$.

3.3 Repeat for x = 200

Using the updated model, x=200 matches C3 because |200−200|=0 is less than 2.5×10=25. It does not match C1 or C2. The ranking remains C1, C2, C3 because their updated w/σ values are approximately 0.158, 0.0475 and 0.0095. The first two components already accumulate to 0.905, exceeding T=0.7. Therefore C3 is outside the selected background portion and x=200 is classified as foreground, even though it matches a Gaussian component.

Update weights: w1=0.62×0.95=0.589; w2=0.285×0.95=0.27075; w3=0.095×0.95+0.05=0.14025. For C3, μ remains 200 and variance becomes 0.95×100=95 because the observed value equals its mean; σ≈9.747. Component 2 (mean 60) can represent a stable dark background region such as a shadowed part of the scene. 'Background' is a statistical model of recurring pixel values, not a semantic label meaning bright or empty.

3.4 Compare the three background models

GMM stores multiple distributions per pixel and can represent multimodal backgrounds such as moving leaves or changing reflections. A running average stores one evolving mean and is simpler, but it cannot represent multiple recurring modes well. Frame differencing compares adjacent frames and reacts quickly to motion, but static objects disappear from the foreground result and camera noise causes false motion.

3.5 Stopped-vehicle protection

A vehicle waiting at a red light may gradually be absorbed into an adaptive background. Maintain object tracks from the detector and a stopped-duration timer. If a tracked vehicle remains in the same region for a configured period, freeze or greatly reduce the background update only inside its protected mask. Allow slow global adaptation elsewhere so dusk changes can still be learned. Release the protection when the vehicle moves away. The freeze must be spatially limited; otherwise the model will fail to adapt to lighting.

Three-Technique Comparison

Technique	Strengths	Limitations / likely failure	Best role

GMM	Handles multiple background modes; adapts online; can represent swaying trees and repetitive changes.	May absorb stopped vehicles; shadows and camera shake can create foreground noise; parameter-sensitive.	Preferred classical background model with track-aware protection.
Running average	Simple, low memory and low computation; smooth adaptation.	Single mean cannot represent multiple modes; may absorb stationary objects; shadows cause errors.	Lightweight baseline for stable scenes.
Frame differencing	Very simple and immediate motion detection; no background training.	Misses stopped objects; sensitive to noise, illumination change and camera motion.	Useful for sudden-motion baseline, not as the only background model.

Algorithmic Pseudocode

```
ALGORITHM ProtectedBackgroundUpdate
INPUT: current frame, GMM model, tracked vehicle masks
OUTPUT: foreground mask and updated GMM

1. Detect and track vehicles in the current frame.
2. For each pixel:
   a. If pixel lies inside a confirmed stopped-vehicle mask:
      reduce learning rate to a very small value or freeze its update.
   b. Otherwise:
      update the GMM using the normal learning rate.
3. Generate the foreground mask using the selected background components.
4. Remove isolated noise with a small morphological operation.
5. If the tracked vehicle moves, remove its protected mask.
6. Continue normal adaptation outside protected regions to follow dusk changes.
END ALGORITHM
```

Evidence to Record and Decision

Record mask precision/recall, shadow false positives, stopped-vehicle persistence, dusk adaptation and CPU cost. Justify K, α, T and the matching threshold. The hand-calculated values above follow the specified update equations.

Experimental reporting note: Fill the result columns from the assigned video/images and actual program execution. Do not present predicted values as measured results.

TASK 4 – Optical Flow and Motion Estimation: Compare Three Methods (6 Marks)

Question / Required Work

Explain brightness constancy and derive $I_x \cdot u + I_y \cdot v + I_t = 0$; discuss the aperture problem and Lucas–Kanade, Horn–Schunck and block matching. Calculate speed for 9 pixels/frame at 25 fps when 5 m corresponds to 150 pixels, including ±1 pixel displacement error; compare methods against manual tracks and compare with Farneback/RAFT.

Compare object recognition techniques (optical flow, motion estimation) · motion characteristics (velocity of objects)

Technique	**Basic idea and key OpenCV functions**
A. Lucas–Kanade	Local window least squares on the flow constraint, usually with an image pyramid for large motion.
B. Horn–Schunck	Global method: flow constraint plus a smoothness term over the whole image, solved iteratively.
C. Block matching	Motion estimation by searching a window for the block with the smallest SAD; block size and search range.

- Analyze: state the brightness-constancy assumption and derive the optical-flow constraint $I_x \cdot u + I_y \cdot v + I_t = 0$. Explain the aperture problem and how A, B and C each resolve it.
- Hand calculation: a vehicle moves 9 pixels per frame at 25 frames per second. Two road markings 5 m apart are 150 pixels apart in the image. Compute its speed in m/s and km/h, and the speed error caused by a ±1 pixel error in displacement. State the assumptions behind this conversion.
- Compare A, B and C on the crossing video for one vehicle and one pedestrian against your manually tracked positions over at least 10 frames: displacement error, speed error and computational cost.
- Compare Lucas–Kanade and Farneback with a deep optical-flow network (RAFT) on fast motion and low-texture road surfaces.

- Evidence Expected: Derivation and aperture-problem explanation, the speed calculation with error analysis, a three-method comparison against manual tracking, and a table of the what-if results.

Answer

4.1 Optical-flow constraint derivation

Brightness constancy assumes that a moving image point keeps approximately the same intensity between two nearby frames: $I(x,y,t)=I(x+u\Delta t,y+v\Delta t,t+\Delta t)$. Using a first-order Taylor expansion and subtracting $I(x,y,t)$, we obtain $I_x \cdot u\Delta t + I_y \cdot v\Delta t + I_t\Delta t \approx 0$. Dividing by Δt gives $I_x \cdot u + I_y \cdot v + I_t = 0$, where (u,v) is image velocity.

This is one equation with two unknowns, u and v. The aperture problem occurs when motion is observed only through a small local edge: motion along the edge cannot be uniquely determined. Additional assumptions are required.

4.2 How the three methods address motion

Lucas–Kanade assumes nearly constant flow in a small local window and solves a least-squares system using multiple pixels. An image pyramid helps handle larger displacement. It is efficient for sparse points but may fail in textureless areas or under large motion if the pyramid/track settings are poor.

Horn–Schunck solves a global optimization problem combining the optical-flow constraint with a smoothness term that encourages neighbouring pixels to have similar flow. It produces dense flow but can oversmooth motion boundaries and is sensitive to brightness changes.

Block matching searches a neighbourhood in the next frame for the block with minimum matching cost, commonly SAD or SSD. It is conceptually simple but depends on block size and search range; repeated road textures and occlusion can create incorrect matches.

4.3 Hand calculation of vehicle speed

Image scale = 150 pixels / 5 m = 30 pixels per metre. Vehicle displacement = 9 pixels/frame × 25 frames/s = 225 pixels/s. Physical speed = 225/30 = 7.5 m/s. In km/h: 7.5×3.6 = 27 km/h.

With ±1 pixel/frame displacement uncertainty, the displacement may be 8 or 10 pixels/frame. At 8 pixels/frame: speed = (8×25)/30 = 6.667 m/s = 24 km/h. At 10 pixels/frame: speed = (10×25)/30 = 8.333 m/s = 30 km/h. Therefore the estimated error is ±3 km/h under this scale. Assumptions: the object moves approximately in the calibrated road plane, the pixel-to-metre scale is valid at that location, the camera is fixed, frame rate is known, and measured displacement follows the same physical point on the object.

4.4 Comparison and deep flow

Evaluate one pedestrian and one vehicle using manually tracked positions for at least 10 frames. Report endpoint/displacement error, speed error and time. Farneback is a classical dense optical-flow method that uses polynomial expansion and is often a useful OpenCV baseline. RAFT is a learned deep optical-flow network that can handle challenging motion more accurately in some cases, but model size, hardware and inference latency may make it unsuitable for CPU-only real-time use. Low-texture asphalt and dusk violate or weaken brightness constancy, while motion blur and occlusion cause mismatches.

Three-Technique Comparison

Technique	Strengths	Limitations / likely failure	Best role
Lucas–Kanade	Efficient sparse flow; tracks selected corners; pyramids support larger motion.	Needs textured points; local constant-flow assumption fails at boundaries/occlusion.	Track detected pedestrian/vehicle points between detector frames.
Horn–Schunck	Dense flow; global smoothness regularization fills low-texture gaps.	Can oversmooth object boundaries; computationally heavier; brightness-constancy sensitive.	Dense motion visualization and broad motion field.
Block matching	Simple search and interpretable SAD/SSD cost.	Sensitive to block size, repeated texture, occlusion and search range.	Low-cost baseline for approximately translational motion.

Algorithmic Pseudocode

```
ALGORITHM EstimateObjectSpeed
INPUT: tracked image positions p(t), p(t+1), frame rate F, local scale S pixels/metre
OUTPUT: speed in m/s and km/h

1. Compute displacement_pixels = EuclideanDistance(p(t), p(t+1)).
2. Compute velocity_pixels_per_second = displacement_pixels × F.
3. Obtain calibrated local scale S for the object's road-plane location.
4. If S is invalid or the object is not on the calibrated plane:
      mark metric speed as unavailable.
   Else:
      speed_mps = velocity_pixels_per_second / S.
      speed_kmh = speed_mps × 3.6.
5. Smooth speed across several valid tracked frames.
6. Report uncertainty caused by pixel displacement and calibration error.
END ALGORITHM
```

Evidence to Record and Decision

Record manual-versus-estimated displacement error and speed error for at least 10 frames, plus processing time. Report separate results for pedestrian and vehicle. Include a what-if table for blur, low texture and large motion.

Experimental reporting note: Fill the result columns from the assigned video/images and actual program execution. Do not present predicted values as measured results.

TASK 5 – Stereo Vision and Structure from Motion: Compare Three Depth Techniques (7 Marks)

Question / Required Work

Calculate stereo depth for f=700 pixels, B=0.12 m and disparities 14, 13, 3 and 2 pixels; analyze one-pixel depth sensitivity; find baseline for $\Delta Z \leq 0.5$ m at Z=20 m and $\Delta d = 0.5$ px. Compare block matching, SGM and structure from motion; design nearest-pedestrian distance estimation and compare with monocular depth, NeRF and 3D Gaussian splatting.

Compare object and pattern recognition techniques (stereo vision, structure from motion) · spatial characteristics (depth) and motion characteristics (camera motion)

Part (a) – Analyze by hand. A rectified stereo pair has focal length f = 700 pixels and baseline B = 0.12 m.

- Compute the depth $Z = fB/d$ for disparities d = 14, 13, 3 and 2 pixels. Compare the depth change caused by a one-pixel disparity error at near range and at far range. Explain the non-linearity.
- Using $\Delta Z \approx Z^2 \cdot \Delta d/(f \cdot B)$, find the baseline needed to achieve $\Delta Z \leq 0.5$ m at Z = 20 m with Δd = 0.5 pixel. Comment on whether such a rig is practical at the gate.

Part (b) – Compare.

Technique	**Basic idea and key OpenCV functions**
A. Block-matching stereo	Rectify the pair, then match windows along each row using SAD/SSD; disparity gives depth.
B. Semi-global matching (SGM)	Pixel-wise matching cost aggregated along several image paths with smoothness penalties.
C. Structure from motion	From a moving-camera clip, track features, estimate camera motion and a sparse 3D structure (correct only up to scale).

- Compare A, B and C on your known distances: depth error at three points, density of the result, failure cases (textureless road, occlusion at edges, repetitive railings, reflective cars), calibration needs and speed.

- For C, explain why reconstruction is correct only up to an unknown scale, and how motion between frames plays the role of the stereo baseline.

Part (c) – Create.

- Design a procedure (pseudocode) that reports the distance to the nearest pedestrian by combining a detector box (Task 2) with the depth result of your chosen technique. Decide which disparity/depth statistic inside the box you use, how invalid values are handled, and justify the choices.
- Compare stereo depth with a monocular depth network (for example, Depth Anything), and explain how NeRF and 3D Gaussian splatting reconstruct a scene from many views.

Evidence Expected: *Hand-calculation tables, a three-technique comparison against known distances, failure examples, the scale-ambiguity explanation and the distance-to-pedestrian pseudocode.*

Answer

5.1 Stereo depth hand calculation

Stereo depth is $Z=fB/d$. Here $fB=700\times0.12=84$ pixel-metres. Therefore: $d=14$ gives $Z=84/14=6$ m; $d=13$ gives $84/13\approx6.46$ m; $d=3$ gives $84/3=28$ m; $d=2$ gives $84/2=42$ m.

Disparity d (px)	Depth Z (m)	Change from previous disparity
14	6.00	—
13	6.46	+0.46 m when disparity falls by 1 px
3	28.00	—
2	42.00	+14.00 m when disparity falls by 1 px

Depth is inversely proportional to disparity, so the same one-pixel error produces a much larger depth error at small disparities (far range). The relationship is nonlinear: $Z=fB/d$ and $dZ/dd=-fB/d^2$.

5.2 Baseline requirement

Using $\Delta Z \approx Z^2 \cdot \Delta d/(fB)$, rearrange to $B \geq Z^2 \cdot \Delta d/(f \cdot \Delta Z)$. Substitute $Z=20$ m, $\Delta d=0.5$ px, $f=700$ px and $\Delta Z=0.5$ m: $B \geq (400 \times 0.5)/(700 \times 0.5) = 200/350 \approx 0.571$ m. A baseline of about 0.57 m is wide for a compact gate-mounted rig and may create synchronization, calibration, occlusion and installation difficulties. The calculation also assumes the disparity uncertainty and calibration are as stated.

5.3 Technique comparison

Block matching rectifies the stereo pair and searches along each epipolar row for a similar window using SAD/SSD. It is relatively simple and fast but can fail on textureless road surfaces, repeated patterns and occlusion boundaries. Semi-global matching (SGM) aggregates matching costs along multiple paths with smoothness penalties; it generally gives denser and smoother disparity but costs more computation. Structure from motion (SfM) tracks features across a moving-camera sequence, estimates camera motion and triangulates sparse 3D points.

5.4 Why SfM has scale ambiguity

Monocular SfM estimates camera motion and scene structure only up to an unknown global scale. If all reconstructed translations and 3D points are multiplied by the same constant, the projected image coordinates remain the same. A known distance, known camera motion or other scale reference is needed to recover metric units. Camera movement between frames acts like a changing baseline, while stereo uses a fixed calibrated baseline.

5.5 Nearest-pedestrian distance

Use the detector box to select the pedestrian region in the depth map. Exclude invalid disparity and obvious background pixels; use a robust statistic such as the 20th percentile of valid depth values in the lower-middle portion of the box, rather than the minimum, which may be a noisy outlier. The lower portion is often closer to the road contact region, but this assumption must be checked. If too few valid pixels remain, return 'distance unavailable' instead of inventing a value.

5.6 Monocular depth and novel-view reconstruction

A monocular depth network such as Depth Anything estimates relative or metric-like depth from one image depending on training and calibration; it does not require a stereo pair, but scale accuracy may be uncertain and domain shift matters. NeRF represents a scene using a neural radiance field and synthesizes views by volume rendering. 3D Gaussian splatting represents the scene using many explicit 3D Gaussian primitives and renders views efficiently. Both reconstruct appearance/structure from multiple views; neither should be treated as a direct replacement for calibrated metric distance without validation.

Three-Technique Comparison

Technique	Strengths	Limitations / likely failure	Best role
Block matching	Simple stereo disparity; relatively efficient; directly uses calibrated pair.	Weak on textureless surfaces, occlusions, repeated patterns and reflective cars.	Baseline dense depth method.
Semi-global matching	Better smoothness and disparity quality across regions; handles more difficult matching than simple local SAD.	Higher computation and still struggles with textureless/reflective regions.	Preferred classical stereo option if accuracy is more important than minimum latency.
Structure from motion	Uses moving camera and many frames; recovers sparse 3D structure without a stereo rig.	Scale ambiguous; feature tracking and camera motion required; dynamic objects complicate reconstruction.	Scene mapping/calibration support, not the primary instantaneous pedestrian-range estimate.

Algorithmic Pseudocode

```
ALGORITHM NearestPedestrianDistance
INPUT: pedestrian bounding box, disparity/depth map, calibration f and B
OUTPUT: estimated distance or unavailable status

1. Crop the depth map to the pedestrian bounding box.
2. Select a lower-middle region likely to contain the pedestrian's road-contact area.
3. Remove invalid, zero and out-of-range depth values.
4. If the number of valid pixels is below the minimum:
     return DISTANCE_UNAVAILABLE.
5. Compute a robust depth statistic, such as the 20th percentile.
6. Check the result against the calibrated operating range.
7. Return the distance with confidence and validity status.
END ALGORITHM
```

Evidence to Record and Decision

Report depth errors at three known points, disparity density, runtime and failure cases. Include the hand-calculation table and a justified distance statistic for the pedestrian box.

Experimental reporting note: Fill the result columns from the assigned video/images and actual program execution. Do not present predicted values as measured results.

TASK 6 – Pattern Recognition: Compare Three Classifiers on Image-Analysis Features (4 Marks)

Question / Required Work

Extract area, aspect ratio, height, solidity and speed from foreground blobs and classify pedestrian, two-wheeler and four-wheeler using k-NN, SVM and decision tree. Compare confusion matrices, class precision/recall, training/prediction time, feature scaling and interpretability; design a cost-sensitive rule where a missed pedestrian costs ten times a false alarm.

Compare pattern recognition techniques (image analysis and classification) · spatial characteristics (shape, size) and motion characteristics (speed) from the foreground blobs of Task 3, extract image-analysis features (area, aspect ratio, height, solidity, and speed from Task 4) and classify each blob as pedestrian, two-wheeler or four-wheeler.

Technique	**Basic idea and key OpenCV functions**
A. k-Nearest Neighbors	Classifies a blob by the majority class among its k closest training samples in feature space.
B. Support Vector Machine	Finds a maximum-margin boundary between classes, optionally with a kernel.
C. Decision tree	Learns a sequence of feature thresholds; easy to read and explain.

- Compare A, B and C using a confusion matrix and per-class precision and recall on frames not used to choose thresholds, plus training time, prediction time, sensitivity to feature scale and interpretability.
- Create: design a cost-sensitive decision rule (pseudocode) in which a missed pedestrian costs ten times a false alarm, and choose the operating threshold from the evidence.
- Compare your classifiers on hand-crafted features with the same classifiers on features from a pretrained network (transfer learning).
- Evidence Expected: Feature table, confusion matrices and held-out precision/recall for the three classifiers, the cost-sensitive rule and its justification.

Answer

6.1 Feature extraction

Use the foreground blob's contour and bounding box. Area is the number of foreground pixels or contour area. Aspect ratio is width/height. Height and width are measured in pixels, preferably normalized by image position or perspective scale. Solidity is contour area divided by convex-hull area; it indicates how filled/compact a shape is. Speed is obtained from Task 4 tracking. These features are low-dimensional and interpretable but may vary with distance and occlusion.

6.2 Classifier behaviour

k-NN stores training samples and assigns the majority label among the k nearest samples. It is easy to implement but prediction cost grows with the training set and feature scaling is essential. SVM learns a maximum-margin decision boundary; with a kernel it can separate nonlinear classes, but parameters and feature scaling require tuning. A decision tree applies a sequence of feature thresholds; it is interpretable and fast but may overfit small training sets.

6.3 Evaluation and cost-sensitive decision

Use a held-out test set that was not used to choose thresholds. A confusion matrix reveals which classes are confused, for example a two-wheeler and rider being classified as a four-wheeler. Report per-class precision and recall, not only overall accuracy. Normalize or standardize features for k-NN and SVM; trees usually do not require scaling.

If a missed pedestrian has cost 10 and a false alarm has cost 1, the decision threshold should prioritize pedestrian recall. One practical rule is to use a lower pedestrian probability threshold and then apply temporal tracking/confirmation to reduce isolated false alarms. The final threshold must be selected on validation data by minimizing an explicit weighted cost, not guessed.

6.4 Hand-crafted versus learned features

Hand-crafted blob features are compact and explainable, and can be computed on a CPU. However, they are weak when a person and vehicle overlap or when a blob is fragmented. Features from a pretrained network encode richer appearance and context and can improve classification under pose/lighting changes, but require more computation and memory. A hybrid can use detector class probabilities together with motion/shape features.

Three-Technique Comparison

Technique	Strengths	Limitations / likely failure	Best role
k-NN	Simple, no complex training; adapts by adding samples.	Sensitive to feature scale and noise; prediction grows with training set.	Small interpretable baseline after feature normalization.
SVM	Strong margin-based classifier; kernels model nonlinear boundaries.	Requires scaling and parameter tuning; less transparent than a small tree.	Good candidate for compact handcrafted features.
Decision tree	Fast inference and human-readable rules; no feature scaling required.	Can overfit and create unstable boundaries from limited data.	Explainable baseline and rule inspection.

Algorithmic Pseudocode

```
ALGORITHM CostSensitiveBlobClassification
INPUT: blob features, trained classifier probabilities
OUTPUT: class label and confidence

1. Obtain class probabilities for pedestrian, two_wheeler and four_wheeler.
2. Set pedestrian acceptance threshold from validation data to prioritize recall.
3. If pedestrian probability exceeds its threshold:
     assign PEDESTRIAN.
4. Else choose the highest-probability vehicle class if it exceeds its threshold.
5. Otherwise assign UNKNOWN and request downstream detector/tracker verification.
6. Evaluate the decision using cost = 10 × pedestrian misses + 1 × false alarms.
END ALGORITHM
```

Evidence to Record and Decision

Report confusion matrices and per-class precision/recall on held-out frames. Include training and inference time. Calculate weighted error cost = 10×missed pedestrians + 1×false alarms (or the stated normalized equivalent) and justify the selected threshold.

Experimental reporting note: Fill the result columns from the assigned video/images and actual program execution. Do not present predicted values as measured results.

TASK 7 – Motion Model for Image Animation: Compare Three Motion Models (5 Marks)

Question / Required Work

Animate a still photograph using motion measured in Task 4. Compare translation-only, per-object affine and keypoint-based motion models; explain keypoints, local affine motion, dense motion field, occlusion map and warping; provide pseudocode and discuss synthetic labelling, consent and deepfake risks.

Compare pattern recognition techniques (motion model for image animation) · motion characteristics applied to a still imageThe safety committee wants an animated reconstruction of a near-miss event produced from a still photograph and the motion measured in the video (Task 4).

Technique	**Basic idea and key OpenCV functions**
A. Translation-only model	Each object in the photograph is shifted by the measured displacement in every frame.
B. Per-object affine model	Each object is moved with translation, scale and rotation estimated from the driving video.
C. Keypoint-based motion model	Keypoints with local affine motion, a dense motion field, an occlusion map and warping of the source image.

- Analyze the parts of the keypoint-based model in C: keypoints, local (affine) motion, dense motion field, occlusion map and warping.
- Create: animate your still photograph with each model using the motion from Task 4, and give pseudocode for transferring motion from the driving video to the still image.
- Compare A, B and C on warping artifacts, holes where something is uncovered, temporal flicker, and (where a real frame exists) the error between animated and real frame. Justify your choice with this evidence.
- Justify safeguards: how the output is labelled as synthetic, and what restrictions apply to animating identifiable people.

- Explain how today's image-animation and video-generation models (first-order motion models, diffusion video models) create motion from a still image, and the deepfake risk this creates.

Evidence Expected: *Pseudocode for motion transfer, animation frames from the three models compared with real frames, an artifact analysis, a justified choice and the labelling/consent safeguards.*

Answer

7.1 Three motion models

Translation-only moves each object by a displacement vector measured from the driving video. It is simple and preserves object appearance, but cannot represent rotation, scale change or deformation. Per-object affine motion estimates translation, rotation, scale and shear for each segmented object. It is more flexible but requires reliable object masks and can distort an object if one affine transform is applied to a non-rigid person.

Keypoint-based motion uses corresponding keypoints between the source still and driving video. Local affine transforms estimate motion around keypoints; a dense motion field interpolates motion across pixels; an occlusion map estimates which regions become hidden or newly visible; and warping maps source pixels into output positions. It can produce more realistic motion but may create holes, stretching, duplicated limbs or temporal flicker.

7.2 Motion transfer procedure

First identify the pedestrian and vehicle regions in the still photograph. Estimate their motion from the driving clip using Task 4. Associate source-image objects with the corresponding tracked objects, estimate a transform for each object and warp each region. Use an occlusion ordering so a foreground vehicle can cover a background person. Fill newly uncovered regions cautiously; inpainting may invent visual content, so label the result as a reconstruction rather than a factual recording.

7.3 Comparison and evaluation

Compare each model on the same source photograph and time steps. Inspect warping artifacts, uncovered holes, temporal flicker and object-boundary consistency. If a real frame exists at a corresponding viewpoint/time, calculate image error only after alignment and account for the fact that the still and real video may have different viewpoints.

A translation model is the safest baseline; per-object affine is a reasonable compromise for rigid vehicles; keypoint-based motion is more expressive but needs stronger safeguards.

7.4 Synthetic labelling, consent and deepfake safeguards

Every output should carry a visible and persistent label such as 'SYNTHETIC RECONSTRUCTION – NOT CCTV FOOTAGE'. Keep the original photograph and generated output separately, record the generation method and date, and do not present invented motion as evidence of what actually happened. Obtain informed consent before animating identifiable people, especially students. Avoid deceptive speech, identity impersonation or publication that could harm a person. Modern first-order motion models transfer motion using keypoints and learned warping; diffusion video models generate temporally coherent frames from image/text/video conditioning. These capabilities also create deepfake risks through non-consensual or misleading animation.

Three-Technique Comparison

Technique	Strengths	Limitations / likely failure	Best role
Translation-only	Simple, stable and easy to explain; low computation.	Cannot represent rotation, scale, articulation or occlusion changes.	Baseline reconstruction and small motion.
Per-object affine	Represents translation, rotation, scale and shear for each object.	Needs object masks; non-rigid objects may deform unnaturally; uncovered areas remain.	Practical compromise for rigid vehicles and limited motion.
Keypoint-based	Can model local motion, dense warping and occlusion.	Most complex; can introduce holes, stretching, flicker and invented content.	Use only when visual realism is required and outputs are clearly labelled.

Algorithmic Pseudocode

```
ALGORITHM TransferMotionToStill
INPUT: still photograph, driving video, object masks, tracked motion
OUTPUT: labelled synthetic animation

1. Detect and segment objects in the still photograph.
2. Track corresponding objects in the driving video.
3. Estimate translation or affine/keypoint motion for each object.
4. For each output time:
   a. Estimate each object's transform or dense motion field.
   b. Warp the object region.
   c. Apply the occlusion ordering and visibility mask.
   d. Fill uncovered areas conservatively.
5. Add a persistent "SYNTHETIC RECONSTRUCTION" label.
6. Store source image, parameters and generated frames separately.
7. Do not use identifiable people without appropriate consent.
END ALGORITHM
```

Evidence to Record and Decision

Include sample frames for all three models, a table of artifact observations and any aligned frame error. Explain limitations when no matching real frame exists. Record the synthetic label and consent safeguards.

Experimental reporting note: Fill the result columns from the assigned video/images and actual program execution. Do not present predicted values as measured results.

TASK 8 – Integrated Evaluation: Compare Three Complete Pipelines (4 Marks)

Question / Required Work

Compare detection-only, motion-first and hybrid pipelines for detection, speed, distance, classification, FPS, dusk/occlusion robustness and assumptions. Perform ablation, compare predictions with observations, recommend methods per job and add ByteTrack or DeepSORT to measure false alarms per hour.

Compare object and pattern recognition techniques together · interpreting visual data with spatial and motion characteristics

Technique	Basic idea and key OpenCV functions
A. Detection-only pipeline	Your chosen deep detector (Task 2) runs on every frame; speed from box displacement; no background model.
B. Motion-first pipeline	GMM background model (Task 3) + optical flow speed (Task 4) + the blob classifier (Task 6).
C. Hybrid pipeline	GMM-gated detector on foreground regions + optical-flow speed + depth (Task 5) for distance.

- Compare A, B and C on accuracy of each output (detection, speed, distance, classification), speed (fps), robustness to dusk and occlusion, and dependence on assumptions.
- Perform an ablation study on the best pipeline: remove one stage at a time (e.g., the GMM gate, the detector, the flow-based speed) and record the effect on the final outputs.
- Discuss whether observations agree with your predictions in Tasks 1–7 and explain differences. Recommend the technique for detection, speed, distance and animation, defended with your own numbers.
- Add multi-object tracking (for example, ByteTrack or DeepSORT) to your best pipeline and measure the effect on false alarms per hour.
- Evidence Expected: A three-pipeline comparison table, an ablation table, a prediction-versus-observation discussion and a final recommendation per job.

Answer

8.1 Pipeline designs

Pipeline A (detection-only) runs the selected detector on every frame and estimates speed from changes in detected box positions. It is simple and less dependent on background subtraction, but may be computationally expensive. Pipeline B (motion-first) uses GMM foreground masks, optical flow and blob classification. It can be efficient in a fixed camera scene but shadows, swaying trees and stopped objects can corrupt the foreground mask. Pipeline C (hybrid) uses GMM to identify candidate moving regions, a detector to confirm semantic objects, optical flow for speed and depth for distance.

8.2 Fair comparison

Evaluate all pipelines on the same labelled video. Report detection precision/recall, speed error, depth error, classification precision/recall, FPS and false alarms per hour. Robustness should be reported separately for daylight, dusk, occlusion and stopped vehicles. A pipeline may be accurate for detection but weak for metric distance; do not combine these into one unqualified score.

8.3 Ablation study

Start with the best complete pipeline. Remove one stage at a time: remove GMM gating, remove the detector, remove optical-flow speed, remove depth or remove tracking. Record the change in each output metric. If removing a stage improves speed but sharply reduces pedestrian recall, that trade-off must be explicitly discussed.

8.4 Multi-object tracking

ByteTrack associates detections across frames using high- and low-confidence detections, helping maintain tracks through temporary weak detections. DeepSORT combines appearance embeddings with motion-based association. Tracking can suppress duplicate alarms and estimate stable trajectories, but incorrect association during occlusion may merge two people or switch identities. Measure false alarm events per hour before and after tracking, not only raw per-frame detections.

Three-Technique Comparison

Technique	Strengths	Limitations / likely failure	Best role

Detection-only	Semantic object detector each frame; simple output interpretation.	High per-frame compute; speed from boxes may be noisy; no direct depth unless added.	Strong semantic baseline and robust fallback.
Motion-first	GMM + flow + blob classifier; can be efficient on fixed camera.	Shadows, trees, stopped vehicles and camera shake can create false blobs or misses.	Useful for motion proposal and low-cost monitoring.
Hybrid	Combines semantic detector, motion gate, flow and depth.	More stages and parameter interactions; gating can hide true objects if too restrictive.	Preferred system architecture if gating never suppresses pedestrian candidates.

Algorithmic Pseudocode

```
ALGORITHM EvaluateIntegratedPipeline
INPUT: labelled video, ground truth tracks, pipeline configurations
OUTPUT: comparison and ablation metrics

1. Run detection-only, motion-first and hybrid pipelines on the same frames.
2. Measure detection, speed, distance and classification metrics separately.
3. Record FPS and false alarm events per hour.
4. Select the strongest pipeline using safety-weighted validation criteria.
5. For each stage in the selected pipeline:
   a. Disable that stage only.
   b. Re-run the same test video.
   c. Record metric changes and runtime changes.
6. Add multi-object tracking and merge consecutive detections into alarm events.
7. Compare false alarms/hour and pedestrian recall before and after tracking.
END ALGORITHM
```

Evidence to Record and Decision

Provide a pipeline comparison table, ablation table and prediction-versus-observation discussion. Report per-output accuracy and false alarms/hour. Tracking should reduce duplicate events without reducing pedestrian recall.

Experimental reporting note: Fill the result columns from the assigned video/images and actual program execution. Do not present predicted values as measured results.

TASK 9 – Re-evaluate and Redesign the System (8 Marks)

Question / Required Work

Redesign for six junctions with a single camera and embedded processor: warn within 200 ms when a pedestrian enters the carriageway while a vehicle approaches above 30 km/h within 15 m; work at night/rain; fewer than one false alarm/hour; no cloud offloading. Compare detector-every-k-frames plus flow tracking, GMM-gated detector and a lighter detector every frame. Include time budget, monocular distance, night/rain impact, compression, experiments, alarm rule and pseudocode.

Compare object and pattern recognition techniques under changed spatial and motion conditions (night, rain, single camera, real-time limit) Changed Requirement: The city decides to install the system at six junctions, each with a single camera and one embedded processor.

- Warn within 200 ms when a pedestrian steps onto the carriageway while a vehicle approaches faster than 30 km/h within 15 m.
- The stereo rig is removed for cost; only one camera is available.
- The system must work at night and in rain.
- False alarms must be fewer than one per hour, to avoid alarm fatigue.

Redesign must stay within the techniques of this unit and run on the embedded processor (no cloud offloading). Simply asking for a faster computer is not acceptable. Compare these three adaptation strategies:

Technique	**Basic idea and key OpenCV functions**
A. Detector every k frames and optical-flow tracking	The heavy detector runs only on every k-th frame; optical flow tracks the boxes in between.
B. GMM-gated detector	The GMM finds foreground regions; the detector runs only on those regions.
C. Lighter detector every frame	A faster, smaller detector (e.g., Viola-Jones or a reduced-resolution compact network) on every frame.

- **9.1 Impact Analysis** **2 Marks•**

  At 25 fps the time available per frame is 40 ms. Measure or estimate the processing time of each stage of your original system and show where the 200 ms budget is lost.

- Without stereo, how will distance be obtained (e.g., from the ground plane and a known camera height, or from object size)? State the assumptions that make this valid.
- How do night and rain affect the GMM, optical flow (brightness constancy) and the detector?Explain how model compression (INT8 quantization, pruning, ONNX or TensorRT export) would let a modern detector meet the 200 ms limit on the embedded processor.
- Evidence Expected: A clear analysis of how each changed condition affects each stage, supported by a time-budget calculation.

9.2 Evaluation of the Original System **2 Marks**

• Which stages remain valid and which become unsafe? Simulate night and rain on your clip (reduced brightness, added noise, streaks) and find which stage fails first.

• What if the thresholds chosen in Tasks 2, 3 and 6 are used unchanged under the new conditions? Test this.

• With a limit of one false alarm per hour, which measure should be controlled first, and what does your Task 8 evidence say about the original system on it?

Evidence Expected: *A technically supported judgement of the original design under the new operating conditions.*

9.3 Compare and Adapt **2 Marks**

• Compare strategies A, B and C on latency, recall and false alarms per hour using your own measurements.

• Decide: retain, modify or redesign. Present the revised system as a flowchart and pseudocode, stating what each stage does and how often it runs, and justify the order.

• Define the alarm rule (distance, speed and number of consecutive frames required) and show how it limits false alarms. If you retain the original design, justify technically why no redesign is necessary.

Evidence Expected: *A comparison of the three strategies, a coherent revised design with pseudocode/flowchart, and the alarm rule with its effect on latency and false alarms.*

9.4 Evidence-Based Defence **2 Marks**

- Defend your decision using the time-budget calculation, your Task 8 results (before and after redesign), the night/rain experiment and the false-alarm limit.
- Where does your redesign give up accuracy or flexibility in exchange for speed or cost? Identify one condition under which it would fail (e.g., a pedestrian hidden behind a bus, a very heavy downpour) and say what you would change next.

Evidence Expected: *A well-supported final decision that follows logically from your own analysis and experiments.Mark Distribution*

Answer

9.1 Impact analysis and 200 ms budget

At 25 fps, one frame arrives every 40 ms. The 200 ms warning requirement spans at most five frame intervals, but the processing and decision path must still be measured end-to-end. Add camera capture, preprocessing, detector, tracking/flow, distance estimation, rule evaluation and alarm output. A pipeline that takes more than 40 ms per frame may still operate asynchronously, but queue growth and stale detections can violate the 200 ms event-to-warning requirement. Measure worst-case and percentile latency, not only average FPS.

Stage	Example budget (illustrative only)	How to measure
Frame capture / decode	5–10 ms	Timestamp frame arrival and decode completion.
Resize / normalization	2–5 ms	Time preprocessing function.
Detector	20–80 ms	Measure model inference on target embedded processor.
Tracking / optical flow	5–20 ms	Measure per-frame association and flow.
Distance / ground-plane estimate	2–10 ms	Measure geometric computation.
Alarm rule / output	1–5 ms	Measure event-to-output delay.

These are planning ranges, not measured results. Replace them with actual target-device measurements. A 200 ms deadline requires the total event path, including confirmation frames, to remain within the limit.

9.2 Monocular distance without stereo

With one camera, estimate road-plane distance using a calibrated ground-plane homography or a known camera height and pitch. If the pedestrian/vehicle contact point is mapped onto the road plane, the image point can be transformed into ground coordinates. This assumes a fixed camera, known camera intrinsics/extrinsics, a locally planar road and a correctly identified contact point. Object-size methods are less reliable because actual object dimensions vary. A monocular depth network can provide a depth prior but should be calibrated against known road distances.

9.3 Night and rain effects

Night reduces illumination and signal-to-noise ratio; rain creates streaks, reflections and blurred edges. GMM may label rain streaks and illumination changes as foreground. Optical flow may fail because brightness constancy is violated by reflections, exposure changes and motion blur. A detector may miss small dark pedestrians or produce false detections from headlights and reflections. Use low-light training/augmentation, exposure-aware preprocessing, temporal tracking and confidence calibration. Do not assume daytime thresholds remain valid.

9.4 Model compression

INT8 quantization reduces weight/activation precision and can reduce memory and inference time, but may affect small-object recall. Pruning removes less important weights or channels and may reduce compute when supported by the runtime. Exporting to ONNX can improve portability and allow optimized execution providers. TensorRT is an NVIDIA inference optimizer for supported hardware; it is relevant only if the embedded processor has compatible NVIDIA hardware. Benchmark the exact exported model and verify pedestrian recall after compression.

9.5 Original system and three strategies

Strategy A runs the heavy detector every k frames and tracks objects with optical flow between detections. It reduces detector load but tracking drift can miss newly entering pedestrians between detector frames. Strategy B uses GMM foreground regions to gate the detector. It can save computation in static scenes, but a pedestrian missed by GMM may never reach the detector; for safety, the gate must be permissive and periodically run full-frame detection. Strategy C runs a smaller detector every frame, potentially giving more consistent updates and lower latency, but may lose accuracy on small pedestrians.

9.6 False-alarm control and alarm rule

With a target of fewer than one false alarm per hour, control false-positive events first without sacrificing pedestrian recall. Use temporal confirmation, track consistency, road-region constraints, direction of motion and distance/speed conditions. Merge repeated frame-level detections into one event. Example alarm rule: trigger only when a pedestrian track is inside the carriageway region for at least two consecutive valid frames AND a vehicle track is approaching with estimated speed above 30 km/h AND calibrated distance is below 15 m. Because the deadline is 200 ms, confirmation must use a small number of frames and be included in the latency budget. If the pedestrian is occluded, retain a short-lived track rather than immediately clearing it.

9.7 Final redesign decision

A practical design is MODIFY/REDESIGN: use a compact detector at reduced but adequate resolution on every frame or at a carefully selected cadence; track detections between inference frames; estimate distance from calibrated ground-plane geometry; use temporal alarm logic; and reserve GMM as an optional proposal cue rather than a hard gate. The final choice between A, B and C must be based on target-device latency, pedestrian recall and false alarms/hour. Night/rain robustness must be tested, not assumed.

9.8 Limitations

The system may fail during very heavy rain, when a pedestrian is fully hidden behind a bus, when the camera is moved, or when the road surface is not planar. A camera movement invalidates the ground-plane calibration; a fully occluded person cannot be detected from the current frame. The next improvement would be camera tamper detection/recalibration, better multi-frame occlusion handling or a second independent sensor if the requirement permits.

Three-Technique Comparison

Technique	Strengths	Limitations / likely failure	Best role
A. Detector every k frames + flow	Reduces heavy detector calls; tracks objects between detections.	New entrants may be missed between detector runs; flow drift accumulates in rain/night.	Good when detector is accurate but too slow; choose k from measured latency and recall.
B. GMM-gated detector	Runs detector on candidate foreground regions, potentially saving compute.	GMM false negatives can suppress a true pedestrian; night/rain and shadows destabilize masks.	Use only as a soft proposal; never hard-reject all regions outside GMM.
C. Lighter detector every frame	Fresh semantic detections each frame; simple scheduling and lower tracking drift.	Compact model may lose small-object recall or local auto-rickshaw recognition.	Strong candidate if embedded benchmark meets latency and safety recall.

Evidence to Record and Decision

Submit (1) original-stage timing, (2) simulated night/rain results, (3) strategy A/B/C latency, recall and false alarms/hour, (4) a revised flowchart and pseudocode, and (5) a clearly defended RETAIN/MODIFY/REDESIGN decision. Report the target hardware and percentile latency. The final design must satisfy both the 200 ms warning and fewer-than-one-false-alarm/hour constraints based on actual tests.

Experimental reporting note: Fill the result columns from the assigned video/images and actual program execution. Do not present predicted values as measured results.

Algorithmic Pseudocode

```
ALGORITHM GateSafetyWarning
INPUT: video frames, camera calibration, road-crossing polygon
OUTPUT: warning event

1. Capture and preprocess the current frame.
2. Run compact pedestrian/vehicle detector according to selected schedule.
3. Update multi-object tracks using detections and optical flow.
4. For each pedestrian track:
   a. Check whether its foot point enters the carriageway polygon.
5. For each approaching vehicle track:
   a. Estimate speed from tracked displacement and calibrated road scale.
   b. Estimate distance using ground-plane mapping or validated monocular depth.
6. If a pedestrian is in the carriageway for the required consecutive frames
   AND an approaching vehicle has speed > 30 km/h
   AND vehicle distance < 15 m:
      raise one warning event for the track pair.
7. Merge repeated detections into the same event to control alarm count.
8. If calibration or depth is invalid, mark distance uncertain and use the
   conservative fallback defined by the safety policy.
9. Record end-to-end latency and event evidence.
END ALGORITHM
```

OVERALL CONCLUSION

For pedestrian-crossing safety, a single technique is not sufficient. A modern detector provides semantic recognition of pedestrians and vehicles; background modelling and optical flow provide complementary motion information; stereo or calibrated monocular geometry provides distance; and a classifier can interpret shape and speed features. The final design should combine these outputs with tracking and a safety-focused alarm rule.

Classical techniques remain useful because they are explainable and often efficient, but their assumptions must be checked under small-object, occlusion, dusk and rain conditions. A hybrid design is recommended only when each stage is validated and the system never allows a restrictive motion gate to suppress a true pedestrian. The final decision must be supported by the measured accuracy, false alarms per hour and end-to-end latency on the intended hardware.

MARK DISTRIBUTION

Task	**Marks**
Task 1 – Viola-Jones Detection	4
Task 2 – YOLO and Deep Learning Detectors	7
Task 3 – Background Modeling Techniques	5
Task 4 – Optical Flow and Motion Estimation	6
Task 5 – Stereo Vision and Structure from Motion	7
Task 6 – Pattern Recognition Classifiers	4
Task 7 – Motion Models for Image Animation	5
Task 8 – Integrated Pipeline Evaluation	4
Task 9 – System Re-evaluation and Redesign	8
TOTAL	50